

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement mesin abstrak

1. Latihan1dan2

Source Code:

```
package main
import (
    "fmt"
    "math/rand"
)
type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    isBalak bool
}
type Dominoes struct {
    domino [28]Domino
    jumlah int
}
func buatDominoes() Dominoes {
    var dominoes Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            dominoes.domino[idx] = Domino{
                sisi1: i,
                sisi2: j,
                nilai: i + j,
                isBalak: i == j,
            }
            idx++
        }
    }
    dominoes.jumlah = idx
    return dominoes
}
func kocokKartu(dominoes *Dominoes) {
    for i := dominoes.jumlah - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        dominoes.domino[i], dominoes.domino[j] = dominoes.domino[j],
        dominoes.domino[i]
    }
}
func ambilKartu(dominoes *Dominoes) Domino {
    if dominoes.jumlah == 0 {
        return Domino{}
    }
}
```

```

        dominoes.jumlah--
        return dominoes.domino[dominoes.jumlah]
    }

func gambarKartu(D Domino, suit int) int {
    if D.sisi1 == suit {
        return D.sisi2
    }
    if D.sisi2 == suit {
        return D.sisi1
    }
    return -1
}

func nilaiKartu(domino Domino) int {
    return domino.nilai
}

func cocok(kartu1, kartu2 Domino) bool {
    return kartu1.sisi1 == kartu2.sisi1 || kartu1.sisi1 == kartu2.sisi2 || kartu1.sisi2
== kartu2.sisi1 || kartu1.sisi2 == kartu2.sisi2
}

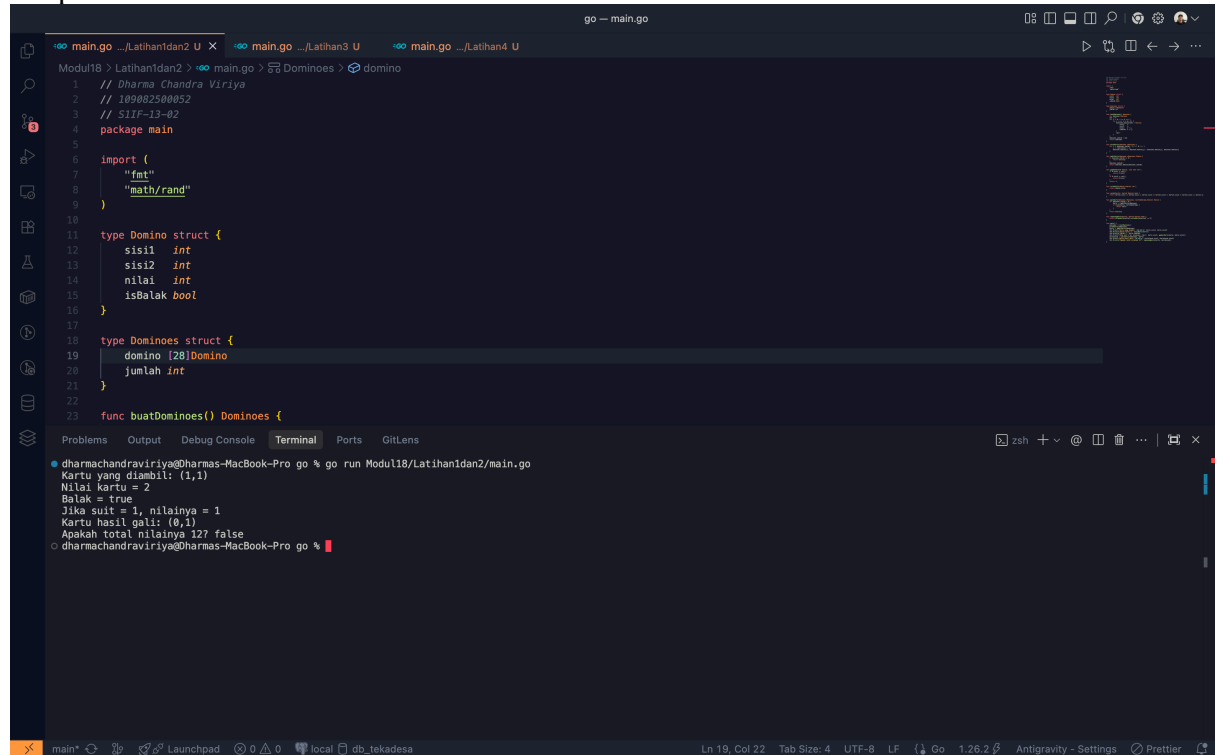
func galiKartu(dominoes *Dominoes, kartuSekarang Domino) Domino {
    for dominoes.jumlah > 0 {
        kartu := ambilKartu(dominoes)
        if cocok(kartu, kartuSekarang) {
            return kartu
        }
    }
    return Domino{}
}

func sepasangKartu(kartu1, kartu2 Domino) bool {
    return nilaiKartu(kartu1)+nilaiKartu(kartu2) == 12
}

func main() {
    dominoes := buatDominoes()
    kocokKartu(&dominoes)
    kartu := ambilKartu(&dominoes)
    fmt.Printf("Kartu yang diambil: (%d,%d)\n", kartu.sisi1, kartu.sisi2)
    fmt.Println("Nilai kartu =", nilaiKartu(kartu))
    fmt.Println("Balak =", kartu.isBalak)
    fmt.Printf("Jika suit = %d, nilainya = %d\n", kartu.sisi1, gambarKartu(kartu,
kartu.sisi1))
    kartuCocok := galiKartu(&dominoes, kartu)
    fmt.Printf("Kartu hasil gali: (%d,%d)\n", kartuCocok.sisi1, kartuCocok.sisi2)
    fmt.Println("Apakah total nilainya 12?", sepasangKartu(kartu, kartuCocok))
}

```

Output:



```
1 // Dharma Chandra Viriya
2 // 109082500052
3 // SIIIF-13-02
4 package main
5
6 import (
7     "fmt"
8     "math/rand"
9 )
10
11 type Domino struct {
12     sisi1 int
13     sisi2 int
14     nilai int
15     isBalak bool
16 }
17
18 type Dominoes struct {
19     domino [28]Domino
20     jumlah int
21 }
22
23 func buatDominoes() Dominoes {
24     // ...
25 }
26
27 func main() {
28     // ...
29 }
30
31 func kocokKartu() {
32     // ...
33 }
34
35 func ambilKartu() {
36     // ...
37 }
38
39 func gambarKartu() {
40     // ...
41 }
42
43 func nilaiKartu() {
44     // ...
45 }
46
47 func cocok() {
48     // ...
49 }
50
51 func galiKartu() {
52     // ...
53 }
54
55 func sepasangKartu() {
56     // ...
57 }
58
59 func main() {
60     // ...
61 }
```

dharmachandraviriyad@Dharmas-MacBook-Pro go % go run Modul18/Latihan1dan2/main.go

Kartu yang diambil: (1,1)

Nilai kartu = 2

Balak = true

Jika suit = 1, nilainya = 1

Kartu hasil gali: (0,1)

Apakah total nilainya 12? false

Deskripsi Program:

Program ini merupakan implementasi mesin abstrak domino menggunakan bahasa Go yang merepresentasikan satu set kartu domino standar berjumlah 28 kartu unik. Struktur data `Domino` digunakan untuk menyimpan nilai kedua sisi kartu (`sisi1` dan `sisi2`), total nilai kartu (`nilai`), serta status balak (`isBalak`). Sementara itu, struktur `Dominoes` digunakan untuk menyimpan seluruh kartu dalam array berukuran 28 elemen dan jumlah kartu yang masih tersedia. Fungsi `buatDominoes()` membangkitkan seluruh kombinasi kartu domino unik dari `(0,0)` hingga `(6,6)` menggunakan perulangan bersarang dan sekaligus menghitung nilai serta status balak setiap kartu. Program menyediakan beberapa operasi dasar mesin domino. Fungsi `kocokKartu()` mengacak urutan kartu menggunakan algoritma Fisher-Yates Shuffle dengan kompleksitas $O(n)$, sedangkan fungsi `ambilKartu()` mengambil kartu dari tumpukan dengan konsep stack (LIFO) sehingga memiliki kompleksitas $O(1)$. Fungsi `gambarKartu()` digunakan untuk menentukan nilai kartu berdasarkan suit yang dipilih, sedangkan `nilaiKartu()` mengembalikan total nilai sebuah kartu. Selain itu, fungsi `cocok()` memeriksa apakah dua kartu memiliki suit yang sama dengan membandingkan seluruh kemungkinan kombinasi sisi kartu. Untuk mendukung operasi permainan domino, fungsi `galiKartu()` melakukan pencarian linear terhadap tumpukan kartu hingga ditemukan kartu yang memiliki suit yang sama dengan kartu acuan, sedangkan fungsi `sepasangKartu()` memeriksa apakah jumlah nilai dua kartu sama dengan 12. Pada fungsi `main()` , program membuat dan mengacak satu set kartu domino, mengambil satu kartu dari tumpukan, menampilkan informasi kartu tersebut, mencari kartu lain yang cocok menggunakan `galiKartu()` , dan akhirnya memeriksa apakah kedua kartu tersebut membentuk pasangan dengan total nilai 12. Dengan demikian, program telah mengimplementasikan operasi-operasi dasar mesin domino sesuai spesifikasi yang diberikan.

2. Latihan3

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    isBalak bool
}

type Dominoes struct {
    domino [28]Domino
    jumlah int
}

func buatDominoes() Dominoes {
    var dominoes Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            dominoes.domino[idx] = Domino{
                sisi1: i,
                sisi2: j,
                nilai: i + j,
                isBalak: i == j,
            }
            idx++
        }
    }
    dominoes.jumlah = idx
    return dominoes
}

func kocokKartu(dominoes *Dominoes) {
    for i := dominoes.jumlah - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        dominoes.domino[i], dominoes.domino[j] = dominoes.domino[j],
        dominoes.domino[i]
    }
}

func ambilKartu(dominoes *Dominoes) Domino {
```

```

        if dominoes.jumlah == 0 {
            return Domino{}
        }
        dominoes.jumlah--
        return dominoes.domino[dominoes.jumlah]
    }

    func gambarKartu(D Domino, suit int) int {
        if D.sisi1 == suit {
            return D.sisi2
        }
        if D.sisi2 == suit {
            return D.sisi1
        }
        return -1
    }

    func nilaiKartu(domino Domino) int {
        return domino.nilai
    }

    func cocok(kartu1, kartu2 Domino) bool {
        return kartu1.sisi1 == kartu2.sisi1 || kartu1.sisi1 == kartu2.sisi2 || kartu1.sisi2
        == kartu2.sisi1 || kartu1.sisi2 == kartu2.sisi2
    }

    func galiKartu(dominoes *Dominoes, kartuSekarang Domino) Domino {
        for dominoes.jumlah > 0 {
            kartu := ambilKartu(dominoes)
            if cocok(kartu, kartuSekarang) {
                return kartu
            }
        }
        return Domino{}
    }

    func sepasangKartu(kartu1, kartu2 Domino) bool {
        return nilaiKartu(kartu1)+nilaiKartu(kartu2) == 12
    }

    func mainGapleh(dominoes *Dominoes) {
        fmt.Println("\n=== SIMULASI GAPLEH ===")
        if dominoes.jumlah == 0 {
            return
        }
        kartuSekarang := ambilKartu(dominoes)
        fmt.Printf("Awal: (%d,%d)\n", kartuSekarang.sisi1, kartuSekarang.sisi2)
        for dominoes.jumlah > 0 {
            kartuBaru := galiKartu(dominoes, kartuSekarang)
            if kartuBaru == (Domino{}) {
                fmt.Println("Tidak ada kartu yang cocok")
                break
            }
        }
    }

```



```

        fmt.Printf(
            "-> (%d,%d)\n",
            kartuBaru.sisi1,
            kartuBaru.sisi2,
        )
        kartuSekarang = kartuBaru
    }
}

func main() {
    dominoes := buatDominoes()
    kocokKartu(&dominoes)
    kartu := ambilKartu(&dominoes)
    fmt.Printf("Kartu yang diambil: (%d,%d)\n", kartu.sisi1, kartu.sisi2)
    fmt.Println("Nilai kartu =", nilaiKartu(kartu))
    fmt.Println("Balak =", kartu.isBalak)
    fmt.Printf("Jika suit = %d, nilainya = %d\n", kartu.sisi1, gambarKartu(kartu,
kartu.sisi1))
    kartuCocok := galiKartu(&dominoes, kartu)
    fmt.Printf("Kartu hasil gali: (%d,%d)\n", kartuCocok.sisi1, kartuCocok.sisi2)
    fmt.Println("Apakah total nilainya 12?", sepasangKartu(kartu, kartuCocok))
    dominoGapleh := buatDominoes()
    kocokKartu(&dominoGapleh)
    mainGapleh(&dominoGapleh)
}

```

Output:

The screenshot shows a Go IDE with a source code editor and a terminal window. The source code is a Go program for a domino game simulation. The terminal output shows the execution results, including the cards drawn, the value of the cards, and the result of the simulation.

```

go -- main.go

Modul18 > Latihan3 > main.go > main
1 // Dharma Chandra Viriya
2 // 109082500052
3 // S1TF-13-02
4 package main
5
6 import (
7     "fmt"
8     "math/rand"
9 )
10
11 type Domino struct {
12     sisi1 int
13     sisi2 int
14     nilai int
15     isBalak bool
16 }
17
18 type Dominoes struct {
19     domino [28]Domino
20     jumlah int
21 }
22
23 func buatDominoes() Dominoes {
24
25     === SIMULASI GAPLEH ===
26     Awal: (3,4)
27     -> (1,4)
28     -> (1,1)
29     -> (1,3)
30     -> (0,3)
31     -> (0,1)
32     -> (0,2)
33     -> (0,4)
34     Tidak ada kartu yang cocok
35     dharmachandraviriy@Dharmas-MacBook-Pro go %

```

Deskripsi Program:

Program ini merupakan pengembangan dari mesin abstrak domino dengan menambahkan simulasi permainan **Gapleh**. Struktur data `Domino` digunakan untuk menyimpan informasi setiap kartu berupa nilai kedua sisi, total nilai kartu, dan status balak, sedangkan struktur `Dominoes` digunakan untuk menyimpan satu set kartu domino yang terdiri dari 28 kartu unik beserta jumlah kartu yang masih tersedia. Fungsi `buatDominoes()` membentuk seluruh kombinasi kartu domino dari `(0,0)` hingga `(6,6)`, sementara `kocokKartu()` mengacak urutan kartu menggunakan algoritma Fisher-Yates Shuffle dan `ambilKartu()` mengambil kartu dari tumpukan dengan konsep stack (LIFO). Program juga menyediakan beberapa operasi dasar domino, seperti `gambarKartu()` untuk menentukan nilai kartu berdasarkan suit yang dipilih, `nilaiKartu()` untuk memperoleh total nilai kartu, `cocok()` untuk memeriksa kesamaan suit antara dua kartu, `galiKartu()` untuk mencari kartu yang memiliki suit yang sama dengan kartu acuan, dan `sepasangKartu()` untuk memeriksa apakah total nilai dua kartu sama dengan 12. Operasi-operasi tersebut digunakan untuk mensimulasikan mekanisme dasar permainan domino sesuai spesifikasi mesin abstrak yang diberikan. Simulasi permainan Gapleh diimplementasikan pada fungsi `mainGapleh()`. Program mengambil satu kartu sebagai kartu awal, kemudian secara berulang mencari kartu lain yang memiliki suit yang sama menggunakan fungsi `galiKartu()`. Setiap kartu yang cocok akan disambungkan ke rangkaian dan menjadi acuan untuk pencarian berikutnya. Proses berlangsung hingga tidak ada lagi kartu yang dapat disambungkan atau tumpukan kartu habis. Dengan pendekatan ini, program berhasil merepresentasikan konsep dasar permainan Gapleh, yaitu menyusun rangkaian kartu domino berdasarkan kesamaan nilai pada sisi yang saling terhubung.

3. Latihan4

Source Code:

```
package main
import (
    "fmt"
)
type MesinKarakter struct {
    data string
    pos  int
}

func start(mesinKarakter *MesinKarakter, teks string) {
    mesinKarakter.data = teks
    mesinKarakter.pos = 0
}

func maju(mesinKarakter *MesinKarakter) {
    if mesinKarakter.pos < len(mesinKarakter.data) {
        mesinKarakter.pos++
    }
}

func eop(mesinKarakter MesinKarakter) bool {
    if mesinKarakter.pos >= len(mesinKarakter.data) {
        return true
    }
    return mesinKarakter.data[mesinKarakter.pos] == '!'
}

func cc(mesinKarakter MesinKarakter) byte {
    return mesinKarakter.data[mesinKarakter.pos]
}

func bacaSemua(teks string) {
    var mesinKarakter MesinKarakter
    start(&mesinKarakter, teks)
    for !eop(mesinKarakter) {
        fmt.Printf("%c", cc(mesinKarakter))
        maju(&mesinKarakter)
    }
    fmt.Println()
}

func hitungKarakter(teks string) int {
    var mesinKarakter MesinKarakter
    start(&mesinKarakter, teks)
    jumlah := 0
    for !eop(mesinKarakter) {
```

```

        jumlah++
        maju(&mesinKarakter)
    }
    return jumlah
}

func hitungA(teks string) int {
    var mesinKarakter MesinKarakter
    start(&mesinKarakter, teks)
    jumlah := 0
    for !eop(mesinKarakter) {
        if cc(mesinKarakter) == 'A' {
            jumlah++
        }
        maju(&mesinKarakter)
    }
    return jumlah
}

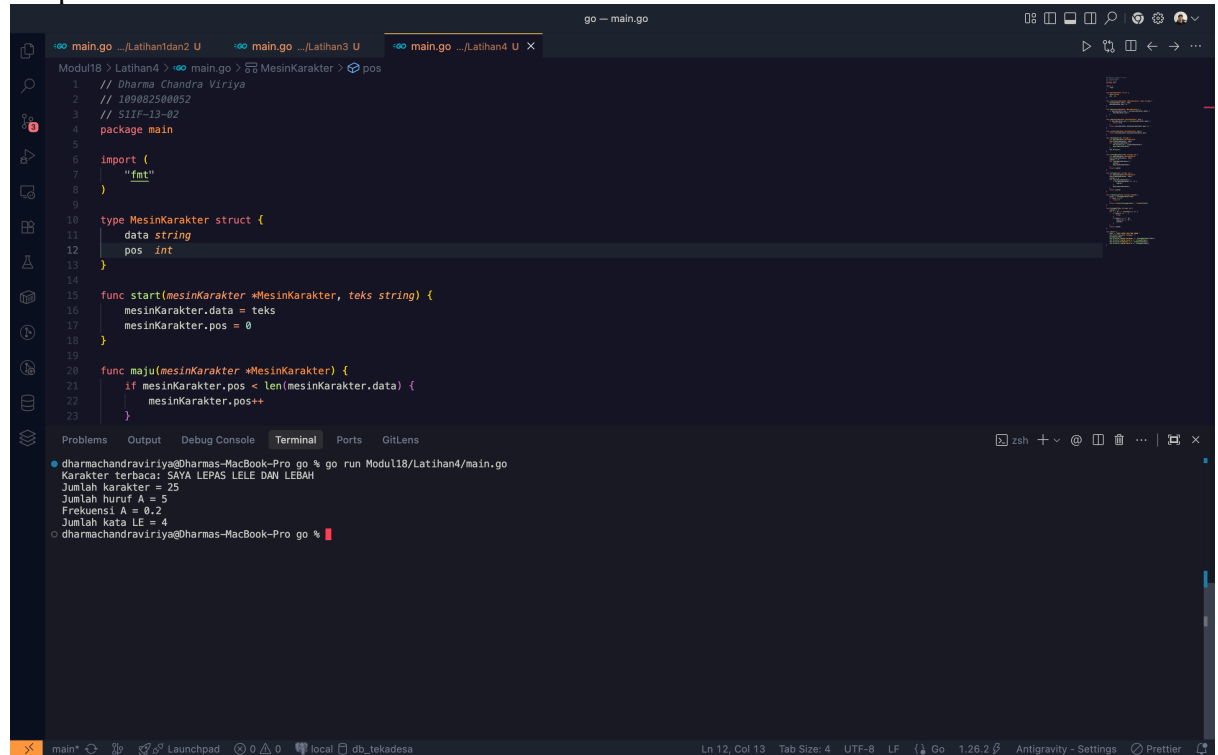
func frekuensiA(teks string) float64 {
    total := hitungKarakter(teks)
    if total == 0 {
        return 0
    }
    return float64(hitungA(teks)) / float64(total)
}

func hitungLE(teks string) int {
    jumlah := 0
    for i := 0; i < len(teks)-1; i++ {
        if teks[i] == '!' {
            break
        }
        if teks[i] == 'L' &&
            teks[i+1] == 'E' {
            jumlah++
        }
    }
    return jumlah
}

func main() {
    teks := "SAYA LEPAS LELE DAN LEBAH."
    fmt.Print("Karakter terbaca: ")
    bacaSemua(teks)
    fmt.Println("Jumlah karakter =", hitungKarakter(teks))
    fmt.Println("Jumlah huruf A =", hitungA(teks))
    fmt.Println("Frekuensi A =", frekuensiA(teks))
    fmt.Println("Jumlah kata LE =", hitungLE(teks))
}

```

Output:



```
1 // Dharma Chandra Viriya
2 // 109002500052
3 // S11F-13-02
4 package main
5
6 import (
7     "fmt"
8 )
9
10 type MesinKarakter struct {
11     data string
12     pos int
13 }
14
15 func start(mesinKarakter *MesinKarakter, teks string) {
16     mesinKarakter.data = teks
17     mesinKarakter.pos = 0
18 }
19
20 func maju(mesinKarakter *MesinKarakter) {
21     if mesinKarakter.pos < len(mesinKarakter.data) {
22         mesinKarakter.pos++
23     }
24 }
25
26 func bacaSemua(mesinKarakter *MesinKarakter) {
27     fmt.Println("Karakter terbaca: " + mesinKarakter.data)
28 }
29
30 func hitungKarakter(mesinKarakter *MesinKarakter) {
31     fmt.Println("Jumlah karakter = " + fmt.Sprintf("%d", len(mesinKarakter.data)))
32 }
33
34 func hitungA(mesinKarakter *MesinKarakter) {
35     count := 0
36     for i := 0; i < len(mesinKarakter.data); i++ {
37         if mesinKarakter.data[i] == 'A' {
38             count++
39         }
40     }
41     fmt.Println("Jumlah huruf A = " + fmt.Sprintf("%d", count))
42 }
43
44 func frekuensiA(mesinKarakter *MesinKarakter) {
45     count := 0
46     for i := 0; i < len(mesinKarakter.data); i++ {
47         if mesinKarakter.data[i] == 'A' {
48             count++
49         }
50     }
51     total := len(mesinKarakter.data)
52     frequency := float64(count) / float64(total)
53     fmt.Println("Frekuensi A = " + fmt.Sprintf("%.2f", frequency))
54 }
55
56 func hitungLE(mesinKarakter *MesinKarakter) {
57     count := 0
58     for i := 0; i < len(mesinKarakter.data)-1; i++ {
59         if mesinKarakter.data[i] == 'L' && mesinKarakter.data[i+1] == 'E' {
60             count++
61         }
62     }
63     fmt.Println("Jumlah kata LE = " + fmt.Sprintf("%d", count))
64 }
65
66 func main() {
67     mesinKarakter := &MesinKarakter{}
68     start(mesinKarakter, "SAYA LEPAS LELE DAN LEBAH")
69     bacaSemua(mesinKarakter)
70     hitungKarakter(mesinKarakter)
71     hitungA(mesinKarakter)
72     frekuensiA(mesinKarakter)
73     hitungLE(mesinKarakter)
74 }
```

Terminal Output:

```
dharmachandravirya@Dharmas-MacBook-Pro go % go run Modul18/Latihan4/main.go
Karakter terbaca: SAYA LEPAS LELE DAN LEBAH
Jumlah karakter = 25
Jumlah huruf A = 5
Frekuensi A = 0.2
Jumlah kata LE = 4
dharmachandravirya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini merupakan implementasi **mesin abstrak karakter** menggunakan bahasa Go untuk membaca dan menganalisis rangkaian karakter yang diakhiri dengan tanda titik (.) sebagai penanda akhir proses (*End of Process/EOP*). Struktur data `MesinKarakter` digunakan untuk menyimpan teks yang akan diproses (`data`) dan posisi karakter yang sedang dibaca (`pos`). Operasi dasar mesin karakter terdiri dari `start()` untuk menginisialisasi mesin pada awal teks, `maju()` untuk memindahkan posisi pembacaan ke karakter berikutnya, `eop()` untuk memeriksa apakah pembacaan telah mencapai tanda titik atau akhir teks, serta `cc()` untuk mengambil karakter yang sedang berada pada posisi pembacaan saat ini. Berdasarkan operasi dasar tersebut, program mengimplementasikan beberapa fungsi analisis karakter. Fungsi `bacaSemua()` digunakan untuk menampilkan seluruh karakter yang dibaca sebelum mencapai tanda titik, sedangkan `hitungKarakter()` menghitung jumlah karakter yang berhasil dibaca. Fungsi `hitungA()` melakukan pencarian dan perhitungan jumlah kemunculan huruf "A" pada teks dengan cara memeriksa setiap karakter menggunakan fungsi `cc()`. Selanjutnya, fungsi `frekuensiA()` menghitung frekuensi kemunculan huruf "A" terhadap seluruh karakter yang terbaca dengan membagi jumlah huruf "A" dengan total karakter yang ada. Selain itu, program juga mengimplementasikan fungsi `hitungLE()` untuk menghitung jumlah kemunculan pasangan karakter "LE" dalam teks. Fungsi ini melakukan penelusuran secara berurutan terhadap setiap karakter dan memeriksa apakah terdapat pola huruf "L" yang langsung diikuti oleh huruf "E". Pada fungsi `main()`, program menggunakan teks "SAYA LEPAS LELE DAN LEBAH." sebagai data masukan, kemudian menampilkan seluruh karakter yang terbaca, jumlah karakter, jumlah huruf "A", frekuensi kemunculan huruf "A", serta jumlah kemunculan pasangan "LE". Dengan demikian, program berhasil merealisasikan operasi-operasi dasar mesin karakter sekaligus melakukan analisis sederhana terhadap sebuah rangkaian teks.

DAFTAR PUSTAKA